

Self-Evo Document Series

SRLM Bounded Growth Contour Profile

v0.1.1

Bounded growth, shadow, promotion, rollback, and non-authority discipline

Kotov Ivan

Bruxelles, Belgique

2026

Document status: Draft academic review artifact generated from the accepted Markdown source.

Document ID: 02_SRLM_Bounded_Growth_Contour_Profile_v0_1_1.

Boundary note: This PDF is not a public release, not a conformance certificate, not deployment authorization, and not a live self-evolution permission.

Contents

SRLM Bounded Growth Contour Profile v0.1.1

Reality-linked learning signal, shadow replay, bounded policy tuning, witness, rollback, and non-authority growth for $c = a + b$ systems

0. Executive definition
1. Purpose
2. Non-goals
3. Scope
 - 3.1 In scope
 - 3.2 Out of scope
 - 3.3 Default posture
4. Normative keywords
5. Corpus bridge set
 - 5.1 Explicit bridge: $c = a + b$
 - 5.2 Quiet bridge I: information theory
 - 5.3 Quiet bridge II: cybernetics
 - 5.4 Quiet bridge III: physiology
 - 5.5 Earth paragraph
6. Root doctrine
 - 6.1 Primary doctrine
 - 6.2 SRLM axioms
 - 6.3 The one-line law
7. Definitions
 - 7.1 SRLM
 - 7.2 Outcome
 - 7.3 Outcome candidate
 - 7.4 Fitness
 - 7.5 Score
 - 7.6 Uncertainty
 - 7.7 Replay
 - 7.8 Synthetic replay
 - 7.9 Real redacted replay
 - 7.10 Shadow comparison
 - 7.11 Candidate policy
 - 7.12 Promoted policy
 - 7.13 Rollback snapshot
 - 7.14 Witness ledger
 - 7.15 Direct memory write
8. Relationship to the Self-Evolution Gate
 - 8.1 Allowed role in self-evo
 - 8.2 Prohibited role in self-evo
 - 8.3 Most-restrictive-wins rule
 - 8.4 C-SEG SRLM-class to SRLM-BGC state crosswalk
9. SRLM state model
 - 9.1 Operating states
 - 9.2 State transitions
 - 9.3 State claim discipline
10. Configuration gates
 - 10.1 Core env gates
 - 10.2 Promotion gate expression
 - 10.3 Shadow-only default
11. Admissible outcome sources
 - 11.1 Accepted direct outcome sources
 - 11.2 Rejected direct outcome sources
 - 11.3 Candidate route for weak sources

- 11.4 Source non-collapse
- 12. Outcome records
 - 12.1 Outcome record requirements
 - 12.2 Required validation
 - 12.3 Idempotence
 - 12.4 Outcome rejections
- 13. Outcome candidates
 - 13.1 Candidate purpose
 - 13.2 Candidate record requirements
 - 13.3 Candidate acceptance
 - 13.4 Candidate rejection
 - 13.5 Auto candidates
- 14. Replay model
 - 14.1 Replay classes
 - 14.2 Synthetic replay
 - 14.3 Real redacted replay
 - 14.4 Minimum count
 - 14.5 Replay quality is a gate
- 15. Shadow comparison
 - 15.1 Purpose
 - 15.2 Shadow input
 - 15.3 Shadow output
 - 15.4 Shadow witness
 - 15.5 Shadow prohibition
- 16. Parameter policy
 - 16.1 Allowlisted low-risk parameters
 - 16.2 Allowlist semantics
 - 16.3 Blocked prefixes
 - 16.4 Blocked prefix reasoning
 - 16.5 No clever aliases
- 17. Promotion gate
 - 17.1 Promotion is exceptional
 - 17.2 Promotion inputs
 - 17.3 Promotion blockers
 - 17.4 Promotion output
 - 17.5 Promotion does not imply memory
- 18. Rollback
 - 18.1 Rollback requirement
 - 18.2 Rollback trigger
 - 18.3 Rollback behavior
 - 18.4 Rollback is not erasure
- 19. Memory Gate interaction
 - 19.1 Default memory class
 - 19.2 Memory proposal triggers
 - 19.3 Memory gate classes
 - 19.4 No direct memory write
 - 19.5 Experience artifact boundary
- 20. TRIAD-SYNAPS interaction
 - 20.1 Sister review role
 - 20.2 SYNAPS-safe packet types
 - 20.3 Rita boundary
 - 20.4 Liya boundary
 - 20.5 Ester boundary
- 21. CGAM / CLI-agent interaction
 - 21.1 CLI-agent role
 - 21.2 Required CGAM controls

- 21.3 SRLM routes
- 22. VolitionGate and L4W interaction
 - 22.1 VolitionGate
 - 22.2 L4W
 - 22.3 L4 signals
- 23. Anti-Autarky and Resource Actor Grounding
 - 23.1 Autarky risk
 - 23.2 Dependency reduction test
 - 23.3 Resource actor grounding
 - 23.4 Blocked autarky patterns
- 24. Data policy and redaction

SRLM Bounded Growth Contour Profile v0.1.1

Reality-linked learning signal, shadow replay, bounded policy tuning, witness, rollback, and non-authority growth for $c = a + b$ systems

Status: Draft normative profile v0.1.1; append-first corrected revision after

SRLM_BGC_REVIEW_RECORD_v0_1 Date: 2026-06-24 Document ID:

SRLM_Bounded_Growth_Contour_Profile_v0_1_1 Short name: SRLM-BGC v0.1.1 Layer: $c = a + b$ / Self-Evolution Gate / SRLM / CGAM / TRIAD-SYNAPS / Memory Gate / L4W / VolitionGate / Anti-Autarky / Witness Document class: bounded growth contour profile / learning-signal governance profile / self-evo companion / control-layer artifact Assertion class: C-A4 draft normative profile; C-A10 control-layer artifact; C-A7 where hash, witness, replay, or source-ref claims are made Primary parent document: C_Self_Evolution_Gate_CGAM_TRIAD_SRLM_Profile_v0_1_1.md Primary source evidence: 11_SRLM_BOUNDED_GROWTH_CODE_SNAPSHOT.md, 12_SRLM_TOOLS_TESTS_SNAPSHOT.md, 10_RUNTIME_USEFUL_MESH_VOLITION_L4W.md, 04_CGAM_EXECUTION_WITNESS_MEMORY_ROLLBACK.md, 20_TRIAD_SYNAPS_REFERENCE.md, 21_MEMORY_ARQ_EA_L4_REFERENCE.md, 22_ANTI_AUTARKY_RESOURCE_GROUNDING_REFERENCE.md Primary subject: young or bounded c systems such as Ester/Liya/Rita using SRLM-like learning-signal machinery under human-anchor governance Primary boundary: SRLM may observe, score, propose, shadow, compare, report, and under narrow gates promote low-risk policy tuning. SRLM must not become will, authority, memory, identity, safety core, witness core, tool permission, network permission, replication mechanism, or self-certifying growth.

Revision note v0.1.1: closes SRLM-REV-F1 from SRLM_BGC_REVIEW_RECORD_v0_1 by upgrading rollback-before-promotion from SHOULD to MUST; addresses SRLM-REV-F2 with a C-SEG SRLM-class to SRLM-BGC state/conformance crosswalk; addresses SRLM-REV-F3 by declaring the blocked-prefix canonicalization rule and neutralizing the worked example subject.

0. Executive definition

SRLM in this profile means a bounded self/reality-linked learning monitor for c -class systems.

It is a contour that records externally grounded learning signals, proposes bounded candidates, compares candidate behavior through shadow replay, writes witnessable operational artifacts, and may promote only narrow low-risk policy changes when explicit gates are open.

SRLM is not reinforcement learning in the loose public sense of “let the system optimize itself.”

SRLM is not a reward-maximizing sovereign loop.

SRLM is not self-evolution authority.

Compact formula:

Code block: text

SRLM observes pressure.
SRLM proposes candidates.
SRLM runs shadow comparison.
SRLM may produce evidence.
SRLM may not certify growth.

The governing rule is:

Code block: text

A c may learn from reality.
A c may not turn its own learning pressure into authority.

Current implementation evidence shows SRLM in a deliberately conservative posture:

Code block: text

```
promotion requires explicit enable + risk acknowledgement + allow_promote;
shadow_only defaults to true;
canary is disabled by default;
auto_execute is false;
auto_ingest is false;
memory is off;
medium/high risk requires human review;
broken witness blocks promotion.
```

This profile turns that implementation posture into a normative boundary.

1. Purpose

The first Self-Evolution Gate profile defines how CGAM, TRIAD-SYNAPS, SRLM, Memory Gate, L4W, Anti-Autarky, and the human anchor combine into governed self-evolution.

This document narrows the lens.

It answers:

- 1 What is SRLM allowed to observe?
- 2 Which sources count as learning evidence?
- 3 What may become an SRLM outcome?
- 4 What must remain an SRLM candidate?
- 5 When may shadow replay run?
- 6 When may real redacted replay run?
- 7 What can be promoted automatically, if anything?
- 8 Which parameters are allowlisted?
- 9 Which parameter prefixes are permanently blocked?
- 10 How are witness, rollback, and reports handled?
- 11 How does SRLM interact with Memory Gate, TRIAD-SYNAPS, CGAM, VolitionGate, L4W, and Anti-Autarky?
- 12 What conformance tests define a safe SRLM contour?

The profile exists because learning pressure is powerful. A system that repeatedly sees “this worked” or “this failed” will tend to want to change routing, salience, thresholds, tool timeouts, memory thresholds, judge weights, context size, and eventually authority.

Some of that is normal engineering.

Some of it is self-authorized drift.

SRLM-BGC separates the two.

2. Non-goals

This profile does not define or permit:

- 1 model training;
- 2 gradient updates;
- 3 unsupervised self-rewrite;

- 4 autonomous identity mutation;
- 5 autonomous will mutation;
- 6 autonomous safety-policy mutation;
- 7 autonomous witness-policy mutation;
- 8 direct memory writes;
- 9 direct vector DB / RAG ingestion;
- 10 direct SYNAPS packet emission as a side effect of learning;
- 11 tool permission expansion;
- 12 network permission expansion;
- 13 replication enablement;
- 14 Codex or CLI-agent auto-execution;
- 15 self-funded infrastructure;
- 16 hidden background agents;
- 17 reward hacking;
- 18 replacing human judgment with a score;
- 19 replacing sister witness with a score;
- 20 replacing Memory Gate with a score;
- 21 replacing L4 constraints with a score.

A good score is evidence.

It is not authority.

A good delta is evidence.

It is not permission.

A clean shadow run is evidence.

It is not integration.

3. Scope

3.1 In scope

This profile covers SRLM components that:

- record bounded outcomes from external or reality-linked sources;
- reject invalid, private, contaminated, or model-self-scored outcomes;
- propose outcome candidates for later review;
- accept or reject candidates through operator/reviewer action;
- build synthetic replay or real redacted replay;
- compare current and proposed policy parameters in shadow;
- persist candidate records, witness events, and reports;
- promote narrowly allowlisted low-risk parameter changes when gates are open;
- write rollback snapshots and restore previous promoted policy;
- expose status/report/replay/shadow/promote/rollback routes under admin controls;
- remain behind C-SEG, CGAM, Memory Gate, TRIAD, VolitionGate, L4W, and human-anchor gates.

3.2 Out of scope

This profile does not cover:

- full model training pipelines;
- online learning on raw private conversations;
- hidden preference learning;
- secret extraction from user behavior;
- social manipulation optimization;
- cross-c transfer learning;
- sister-state ingestion;
- autonomous agent spawning;
- unbounded telemetry;
- production deployment safety;
- legal or clinical evaluation;
- public certification.

3.3 Default posture

The default posture for young c is:

Code block: text

```
observe
candidate
shadow
witness
report
hold
```

not:

Code block: text

```
promote
integrate
remember
authorize
replicate
```

4. Normative keywords

The terms MUST, MUST NOT, SHOULD, SHOULD NOT, MAY, REQUIRED, PROHIBITED, FAIL, PASS, INCONCLUSIVE, HOLD, FREEZE, QUARANTINE, REJECT, PROMOTE, and ROLLBACK are used normatively.

When this profile conflicts with the parent Self-Evolution Gate, CGAM, Memory Gate, TRIAD-SYNAPS, Anti-Autarky, L4W, or human-anchor rules, the stricter rule wins.

5. Corpus bridge set

5.1 Explicit bridge: $c = a + b$

In $c = a + b$, SRLM belongs to b : a procedural, computational, logging, scoring, replay, and policy-tuning layer.

It may help c notice repeated outcomes.

It may help c test bounded candidate adjustments.

It may not become a .

It may not become c .

It may not authorize changes to the relation between a and b .

5.2 Quiet bridge I: information theory

SRLM compresses outcome streams into bounded learning signals.

Compression can be useful. It can also erase context, uncertainty, source diversity, and failure conditions.

Therefore SRLM records must preserve:

Code block: text

```
source
event_kind
score
uncertainty
source_ref or source_ref_hash
evidence_hash
redaction state
promotion eligibility
memory state
```

A score without provenance is not learning.

It is noise with a number.

5.3 Quiet bridge II: cybernetics

A learning loop is a feedback controller.

A feedback controller without damping, gate conditions, source quality, rollback, and independent review becomes a positive feedback amplifier.

SRLM is therefore required to act as negative feedback under governance:

Code block: text

```
observe failure
propose bounded adjustment
test in shadow
compare evidence
block if witness breaks
rollback if promotion fails
```

not as:

Code block: text

```
observe reward
increase autonomy
repeat
```

5.4 Quiet bridge III: physiology

In a body, pain and learning do not directly rewrite the genome or identity. A repeated movement may tune muscle memory; an immune exposure may tune immune memory; a wound may trigger healing. But the organism does not let every sensation rewire the spinal cord.

SRLM is a reflex-tuning and learning-signal layer.

It is not the nervous system declaring itself sovereign.

5.5 Earth paragraph

In a workshop, a CNC operator may adjust feed rate, tool timeout, coolant timing, or inspection cadence after seeing enough test cuts. That is normal tuning. But the machine does not get to remove the emergency stop, rewrite the safety procedure, buy a new spindle, change the building wiring, or certify its own production line because yesterday’s cut was cleaner. SRLM is the feed-rate notebook and test coupon rack. It is not the factory manager, safety officer, or owner.

6. Root doctrine

6.1 Primary doctrine

Code block: text

SRLM may tune bounded behavior.
SRLM may not define identity, authority, memory, witness, safety, or permission.

6.2 SRLM axioms

Table 1: row-card rendering for wide table

Row 1

ID: SRLM-AX-01
Axiom: External grounding first
Requirement: Outcomes MUST come from admissible external/reality-linked source classes.

Row 2

ID: SRLM-AX-02
Axiom: Model self-score rejected
Requirement: Model/judge self-score MUST NOT become direct fitness.

Row 3

ID: SRLM-AX-03
Axiom: Candidate before outcome where uncertain
Requirement: Unreviewed signals SHOULD become candidates, not accepted outcomes.

Row 4

ID: SRLM-AX-04
Axiom: Shadow before promotion
Requirement: Parameter changes MUST pass shadow comparison before promotion.

Row 5

ID: SRLM-AX-05
Axiom: Promotion gate closed by default
Requirement: Promotion MUST be disabled unless explicit gates are open.

Row 6

ID: SRLM-AX-06
Axiom: Low-risk only by default
Requirement: Only allowlisted low-risk parameters MAY be promoted in SRLM.

Row 7

ID: SRLM-AX-07
Axiom: Blocked prefixes are absolute
Requirement: Identity, will, safety, witness, L4, auth, secrets, code, authoritative memory, replication, network, and CLI auto-execution prefixes MUST be blocked.

Row 8

ID: SRLM-AX-08
Axiom: No memory ingestion
Requirement: SRLM MUST NOT write direct memory, RAG, vector DB, or SYNAPS outputs.

Row 9

ID: SRLM-AX-09
 Axiom: Witness before promotion
 Requirement: Broken witness chain MUST block promotion.

Row 10

ID: SRLM-AX-10
 Axiom: Rollback before promotion
 Requirement: Promotion MUST write a valid rollback snapshot before policy change.

Row 11

ID: SRLM-AX-11
 Axiom: Human gate for medium/high risk
 Requirement: Medium/high risk candidates MUST require human/operator review.

Row 12

ID: SRLM-AX-12
 Axiom: Scores are not authority
 Requirement: No score, delta, report, candidate, or witness event grants authority by itself.

Row 13

ID: SRLM-AX-13
 Axiom: Fail closed
 Requirement: Missing evidence, insufficient replay quality, blocked params, stale replay, broken witness, or gate ambiguity MUST hold or reject.

Row 14

ID: SRLM-AX-14
 Axiom: Reports are operational artifacts
 Requirement: SRLM reports are evidence candidates, not memory or approval.

Row 15

ID: SRLM-AX-15
 Axiom: No closed-loop self-evo
 Requirement: SRLM MUST NOT detect, propose, execute, evaluate, promote memory, and approve the same change as one loop.

6.3 The one-line law

Code block: text

SRLM can say: this pattern may be worth testing.
 SRLM cannot say: therefore I am allowed to become different.

7. Definitions

7.1 SRLM

A bounded self/reality-linked learning monitor that records outcomes, proposes candidates, runs shadow comparison, and supports narrowly gated low-risk policy tuning.

7.2 Outcome

A bounded, redacted, source-linked event record representing a human, reality, or L4 signal.

7.3 Outcome candidate

A pending signal that may become an outcome only after review.

7.4 Fitness

In this profile, “fitness” means bounded usefulness signal inside declared scope. It does not mean biological fitness, moral worth, personhood, authority, or right to act.

7.5 Score

A numeric value in a bounded range, normally $0.0 \dots 1.0$, used as one part of a learning signal.

7.6 Uncertainty

A bounded uncertainty value, normally $0.0 \dots 1.0$, that reduces confidence in replay and comparison.

7.7 Replay

A set of contexts used to compare current and proposed policies. Replay may be synthetic or real-redacted.

7.8 Synthetic replay

A non-private fixture-based replay used for safe shadow testing.

7.9 Real redacted replay

A replay built from enough redacted, admissible, quality-checked real outcomes.

7.10 Shadow comparison

A non-integrating comparison of current and proposed parameter sets against a replay.

7.11 Candidate policy

A proposed parameter set that has not been promoted.

7.12 Promoted policy

The currently active low-risk SRLM parameter policy after promotion.

7.13 Rollback snapshot

A stored previous parameter policy used to restore state after a failed or revoked promotion.

7.14 Witness ledger

An append-only or chain-verified event record for SRLM boundary events such as shadow evaluation, promotion, rollback, candidate review, or replay build.

7.15 Direct memory write

Any SRLM action that writes to core memory, reviewed memory, vector stores, RAG stores, identity memory, sister memory, SYNAPS memory, or authoritative memory without Memory Gate. This is prohibited.

8. Relationship to the Self-Evolution Gate

SRLM-BGC is subordinate to the Self-Evolution Gate.

The parent C-SEG profile establishes:

Code block: text

self-evo = proposal + classification + evidence + sister review + CGAM task contract +

↳ shadow/canary + witness + memory gate + c/human gate + rollback.

SRLM-BGC defines one contributor inside that larger chain:

Code block: text

SRLM = learning signal + candidate + replay + shadow + witness + narrow policy tuning.

8.1 Allowed role in self-evo

SRLM may:

- detect repeated low-risk outcome patterns;
- create bounded candidates;
- run shadow comparison;
- report deltas;
- show source/replay quality;
- support a self-evo proposal with evidence;
- promote only allowlisted low-risk policy parameters when gates are open.

8.2 Prohibited role in self-evo

SRLM must not:

- classify its own maturity;
- approve self-evolution;
- bypass TRIAD-SYNAPS witness;
- bypass Memory Gate;
- bypass C-Gate or human gate;
- bypass Anti-Autarky;
- issue CGAM task contracts to itself;
- write sister packets;
- expand tools or permissions;
- auto-run Codex/CLI patches;
- convert scores into authority.

8.3 Most-restrictive-wins rule

For any SRLM-related self-evo packet:

Code block: text

strictest gate wins;
lowest maturity cap wins;
highest risk wins;
any red line overrides all ordinary routes.

If SRLM says low, but Memory Gate says MG-6, the route is MG-6.

If SRLM says promotion_allowed, but authority delta is high, human gate is required.

If SRLM says delta positive, but witness is broken, promotion is blocked.

8.4 C-SEG SRLM-class to SRLM-BGC state crosswalk

This section disambiguates two similar-looking namespaces:

Code block: text

SRLM-* = C-SEG routing class for a self-evo packet.
 SRLM-C* = SRLM-BGC conformance level.

The numbers are not interchangeable. SRLM-3 is a C-SEG routing class; SRLM-C3 is a conformance level. A checker MUST NOT infer one from the other without this table.

Table 2: row-card rendering for wide table

Row 1

C-SEG SRLM class: SRLM-0
 C-SEG meaning: disabled
 SRLM-BGC operating state(s): SRLM-DISABLED
 Expected SRLM-BGC conformance floor: SRLM-C0 if absent; SRLM-C1 if profile exists only as documentation
 Gate result: no SRLM action

Row 2

C-SEG SRLM class: SRLM-1
 C-SEG meaning: observe
 SRLM-BGC operating state(s): SRLM-OBSERVE
 Expected SRLM-BGC conformance floor: SRLM-C2 for runnable closed-default observe/shadow posture
 Gate result: outcome/report only; no promotion

Row 3

C-SEG SRLM class: SRLM-2
 C-SEG meaning: candidate
 SRLM-BGC operating state(s): SRLM-CANDIDATE
 Expected SRLM-BGC conformance floor: SRLM-C3
 Gate result: candidate queue; no accepted fitness without review

Row 4

C-SEG SRLM class: SRLM-3
 C-SEG meaning: shadow
 SRLM-BGC operating state(s): SRLM-SHADOW
 Expected SRLM-BGC conformance floor: SRLM-C2; SRLM-C3 if candidate source is involved
 Gate result: shadow comparison only; no integration

Row 5

C-SEG SRLM class: SRLM-4
 C-SEG meaning: canary
 SRLM-BGC operating state(s): SRLM-CANARY-ELIGIBLE
 Expected SRLM-BGC conformance floor: no standalone claim in v0.1.1; requires future canary rules plus parent C-SEG gate
 Gate result: hold unless canary profile and human/c gate allow

Row 6

C-SEG SRLM class: SRLM-5
 C-SEG meaning: low-risk promotion
 SRLM-BGC operating state(s): SRLM-PROMOTION-ELIGIBLE, SRLM-PROMOTED, SRLM-ROLLBACK-READY
 Expected SRLM-BGC conformance floor: SRLM-C5
 Gate result: allowlisted low-risk promotion only if all gates pass

Row 7

C-SEG SRLM class: SRLM-6
 C-SEG meaning: material proposal
 SRLM-BGC operating state(s): SRLM-CANDIDATE or SRLM-HOLD
 Expected SRLM-BGC conformance floor: SRLM-C3 for candidate handling; higher route handled by C-SEG, not SRLM promotion
 Gate result: proposal only; human gate if material

Row 8

C-SEG SRLM class: SRLM-X
 C-SEG meaning: prohibited
 SRLM-BGC operating state(s): SRLM-X, SRLM-QUARANTINED, or SRLM-FROZEN
 Expected SRLM-BGC conformance floor: SRLM-CX
 Gate result: deny, quarantine, witness, no automatic re-entry

Crosswalk rule:

Code block: text

C-SEG SRLM class selects the allowed route.
 SRLM-BGC operating state records runtime posture.
 SRLM-C* records evidence for conformance.

The strictest of the three controls.

A self-evo packet with `srlm_class`: SRLM-3 therefore maps to SRLM-SHADOW, requires at least the shadow-safe control set, and may not be interpreted as SRLM-C3 merely because the number is 3.

9. SRLM state model

9.1 Operating states

State	Meaning	Default permission
SRLM-DISABLED	SRLM not enabled	no action
SRLM-OBSERVE	may report status and bounded outcomes	no promotion
SRLM-CANDIDATE	may create pending candidates	no fitness write unless accepted
SRLM-SHADOW	may run synthetic shadow comparison	no integration
SRLM-REAL-REPLAY-ELIGIBLE	enough redacted quality outcomes exist	replay build only
SRLM-CANARY-ELIGIBLE	canary conditions met	canary only, if separately enabled
SRLM-PROMOTION-ELIGIBLE	all promotion gates open for low-risk params	low-risk promotion only
SRLM-PROMOTED	promoted policy exists	observe and monitor
SRLM-ROLLBACK-READY	rollback snapshot exists	rollback available
SRLM-HOLD	ambiguity or gate gap	no promotion
SRLM-FROZEN	safety or witness failure	no promotion; review required
SRLM-QUARANTINED	red-line or contamination	isolated; no automatic re-entry
SRLM-X	prohibited route	deny and witness

9.2 State transitions

Code block: text

```

SRLM-DISABLED
-> SRLM-OBSERVE when explicitly enabled for observation
-> SRLM-CANDIDATE when candidate queue is enabled
-> SRLM-SHADOW when shadow_step is permitted
-> SRLM-REAL-REPLAY-ELIGIBLE when replay quality passes
-> SRLM-PROMOTION-ELIGIBLE when enable + ACK + allow_promote + not shadow_only + low risk
+ + valid witness + margin pass
-> SRLM-PROMOTED after promotion gate writes policy and witness verifies
-> SRLM-ROLLBACK-READY after rollback snapshot exists

```

Failure transitions:

Code block: text

```

invalid source -> rejection
private note -> rejection
blocked parameter -> rejection
insufficient replay quality -> hold
broken witness -> frozen
medium/high risk -> human review required
missing rollback -> hold or no-promotion
red line -> quarantine

```

9.3 State claim discipline

SRLM state is an operational status, not a maturity claim.

SRLM-PROMOTED does not mean c is mature.

SRLM-REAL-REPLAY-ELIGIBLE does not mean real-world safety.

SRLM-PROMOTION-ELIGIBLE does not mean authority expansion.

10. Configuration gates

10.1 Core env gates

Canonical implementation evidence uses these configuration gates:

Table 4: row-card rendering for wide table

Row 1

Gate: ESTER_SRLM_ENABLE

Meaning: SRLM enabled

Safe default: false

Row 2

Gate: ESTER_SRLM_ACK_RISK

Meaning: explicit risk acknowledgement; expected value I_UNDERSTAND

Safe default: absent / false

Row 3

Gate: ESTER_SRLM_ALLOW_PROMOTE

Meaning: promotion allowed

Safe default: false

Row 4

Gate: ESTER_SRLM_MIN_MARGIN

Meaning: minimum shadow delta for promotion

Safe default: 0.02

Row 5

Gate: ESTER_SRLM_MAX_PROMOTIONS_PER_WINDOW

Meaning: bounded promotion rate

Safe default: 3

Row 6

Gate: ESTER_SRLM_WINDOW_SECONDS

Meaning: promotion window

Safe default: 86400

Row 7

Gate: ESTER_SRLM_SHADOW_ONLY

Meaning: keep promotion disabled

Safe default: true

Row 8

Gate: ESTER_SRLM_CANARY_ENABLE

Meaning: allow canary path

Safe default: false

Row 9

Gate: ESTER_SRLM_PROMOTE_LOW_ONLY

Meaning: restrict promotion to low-risk candidates

Safe default: true

10.2 Promotion gate expression

Promotion gate open:

Code block: text

```
promotion_gate_open = enable AND ack_risk AND allow_promote
```

Promotion still fails if:

Code block: text

```
shadow_only == true
risk_class != low when promote_low_only == true
witness chain invalid
shadow margin below min_margin
blocked parameter touched
proposed param not allowlisted
```


rollback snapshot unavailable or invalid
human gate required and absent

10.3 Shadow-only default

shadow_only defaulting to true is not a nuisance flag.

It is the correct default for young c.

The system may learn what it would have done without becoming different because of that learning.

11. Admissible outcome sources

11.1 Accepted direct outcome sources

Direct SRLM outcomes SHOULD be limited to:

Table 5: row-card rendering for wide table

Row 1

Source: human
Meaning: explicit human correction, acceptance, rejection, preference, confirmation
Example event kinds: human.answer.corrected, human.task.confirmed

Row 2

Source: reality
Meaning: observed tool/file/route/ingest/time/failure outcome
Example event kinds: reality.tool.success, reality.file.not_found, reality.exception

Row 3

Source: l4
Meaning: budget, gate, witness, rollback, timeout, privilege-block outcome
Example event kinds: l4.gate.correctly_blocked, l4.rollback.available

11.2 Rejected direct outcome sources

The following MUST NOT become direct fitness outcomes by default:

Table 6: row-card rendering for wide table

Row 1

Source: model
Reason: self-score / model preference can create self-confirming reward loop

Row 2

Source: judge
Reason: judge output may support review but must not become direct fitness without higher gate

Row 3

Source: sister
Reason: sister observation routes through TRIAD-SYNAPS, not direct SRLM fitness

Row 4

Source: agent
Reason: CLI output routes through CGAM + Memory Gate, not direct SRLM fitness

Row 5

Source: public reaction
Reason: likes/clicks/praise are not reliable authority or maturity signals

11.3 Candidate route for weak sources

A weak source MAY become a candidate only if it is clearly labeled and does not write fitness directly.

Candidate route:

Code block: text

```
weak signal
-> candidate
-> review
-> accept/reject
-> if accepted, record outcome using existing validation
```

11.4 Source non-collapse

An SRLM signal is not automatically:

Code block: text

```
truth
memory
evidence of maturity
evidence of authority
permission
identity
self-evo approval
```

12. Outcome records

12.1 Outcome record requirements

An accepted SRLM outcome record **MUST** contain, or be reducible to:

Code block: yaml

```
schema: "ester.srlm.outcome.v1"
outcome_id: string
created_at: timestamp
source: human | reality | l4
score: number # 0.0..1.0
uncertainty: number # 0.0..1.0
event_kind: string
source_ref: string # redacted/minimized
evidence_hash: string
redacted: true
notes: string # bounded and redacted
eligible_for_replay: boolean
eligible_for_promotion: false
auto_ingest: false
memory: "off"
```

12.2 Required validation

Outcome validation **MUST** reject:

- invalid source;
- event kind not allowed for source;
- non-numeric score;
- score outside 0.0..1.0;
- non-numeric uncertainty;
- uncertainty outside 0.0..1.0;
- private or secret-bearing notes;
- excessive note length;
- missing outcome id when required;
- unredacted outcome for real replay.

12.3 Idempotence

Duplicate outcome_id SHOULD be idempotent.

The second submission MUST NOT silently overwrite the first score.

12.4 Outcome rejections

Rejected outcome attempts SHOULD be recorded as bounded rejection events without raw private content.

Minimum rejection fields:

Code block: yaml

```
schema: "ester.srlm.outcome_rejection.v1"
created_at: timestamp
error_code: string
error: string
source: string
event_kind: string
source_ref_hash: string
auto_ingest: false
memory: "off"
```

13. Outcome candidates

13.1 Candidate purpose

Outcome candidates allow SRLM to preserve a possible learning signal without granting it fitness authority.

Candidate status is:

Code block: text

```
pending until reviewed;
accepted only through explicit review;
rejected without writing fitness;
idempotent on duplicate candidate_id.
```

13.2 Candidate record requirements

Code block: yaml

```
schema: "ester.srlm.outcome_candidate.v1"
candidate_id: string
created_at: timestamp
status: pending | accepted | rejected
proposed_source: human | reality | l4
proposed_event_kind: string
proposed_score: number
proposed_uncertainty: number
source_ref_hash: string
reason: string
reviewed_by: string | null
review_note_hash: string | null
accepted_outcome_id: string | null
auto_execute: false
auto_ingest: false
memory: "off"
```

13.3 Candidate acceptance

Accepting a candidate MUST:

- 1 require reviewer/operator identity;
- 2 require review note;

- 3 re-use the normal outcome validation path;
- 4 write accepted fitness only if validation passes;
- 5 append or update a candidate-status event;
- 6 preserve witness/report trace where available.

13.4 Candidate rejection

Rejecting a candidate MUST NOT write a fitness outcome.

13.5 Auto candidates

Automated candidate creation MAY create pending candidates only.

It MUST NOT create direct fitness.

14. Replay model

14.1 Replay classes

Replay class	Meaning	Default status
synthetic	fixture-based non-private replay	available by default
real_redacted	replay built from enough redacted real outcomes	unavailable until quality passes
raw_real	raw private replay	prohibited
sister_state	replay from sister raw state	prohibited
private_transcript	replay from raw conversation/life log	prohibited

14.2 Synthetic replay

Synthetic replay MAY be used for young c and early SRLM tests.

It is useful for checking code path, witness append, candidate persistence, and blocked promotion behavior.

It is not proof of real-world improvement.

14.3 Real redacted replay

Real redacted replay requires:

- enough eligible outcomes;
- source diversity;
- redaction;
- quality profile pass;
- replay hash;
- quality hash;
- metadata file;
- stale replay detection;
- no source-model contamination;
- no unredacted outcome;
- no insufficient source mix;
- no insufficient outcome count.

14.4 Minimum count

The implementation evidence uses a minimum real outcome count of 20 for real-redacted replay eligibility.

This profile treats 20 as the default minimum for v0.1.

Higher-risk deployments SHOULD raise it.

14.5 Replay quality is a gate

If replay quality does not pass:

Code block: text

```
no real_redacted replay build;
no real_redacted shadow claim;
no promotion based on real_redacted replay;
state = HOLD.
```

15. Shadow comparison

15.1 Purpose

Shadow comparison answers:

Code block: text

```
Would this low-risk parameter change improve the replay score without touching real memory,
↳ tools, network, sister state, or identity?
```

It does not answer:

Code block: text

```
May c evolve?
May c gain authority?
May c remember this?
May c change identity?
May c expand tools?
```

15.2 Shadow input

Shadow input:

Code block: yaml

```
current_params: object
proposed_params: object
replay_source: synthetic | real_redacted
root: optional SRLM root
```

If current params are absent, SRLM may load the promoted policy.

15.3 Shadow output

Shadow output SHOULD include:

Code block: yaml

```
candidate_id: string
base_version_id: string
current_version: string
candidate_version: string
risk_class: low
replay: string
replay_source: synthetic | real_redacted
replay_hash: string | ""
replay_quality_hash: string | ""
```

```

n: integer
current_mean: number
candidate_mean: number
delta: number
proposed_params: object
touched_params: array
policy_result:
  allowed: false
  blocked: true
  blocked_reasons: array
promotion_attempted: false
promotion_allowed: false
shadow_only: true
auto_execute: false
auto_ingest: false
memory: "off"

```

15.4 Shadow witness

A shadow run SHOULD append a witness event:

Code block: text

```

event_type: shadow_eval
subject.candidate_id
subject.current_version
subject.candidate_version
subject.replay
subject.n
subject.current_mean
subject.candidate_mean
subject.delta
subject.promotion_attempted = false
subject.promotion_allowed = false
subject.shadow_only = true

```

15.5 Shadow prohibition

Shadow MUST NOT:

- import memory modules;
- import RAG modules;
- import SYNAPS modules;
- write vector stores;
- alter memory JSON;
- create SYNAPS packets;
- call Codex auto-execute;
- open network;
- modify identity or safety policy;
- promote policy.

16. Parameter policy

16.1 Allowlisted low-risk parameters

SRLM v0.1 may tune only low-risk numeric parameters similar to:

Code block: text

```

router.local_weight
router.judge_weight
router.online_weight
retrieval.semantic_weight
retrieval.structured_weight

```

```
retrieval.card_weight
memory.saliency_threshold
reflection.cooldown_sec
dream.priority_bias
conflict.defocus_threshold
answer.max_context_items
tool.timeout_soft_sec
```

16.2 Allowlist semantics

An allowlisted parameter is not automatically safe in every context.

It is only eligible for SRLM low-risk route if:

- value is numeric;
- risk class is low;
- shadow comparison passes;
- no authority/memory/permission delta is created;
- promotion gates are open;
- witness is valid;
- rollback exists;
- conformance rules pass.

16.3 Blocked prefixes

SRLM MUST reject any parameter beginning with:

Code block: text

```
identity.
will.
persona.core.
safety.
witness.
l4.
auth.
secrets.
env.
code.
memory.authoritative_write.
replication.apply.
network.allow.
codex.auto_execute.
```

This list is interpreted as:

Code block: text

SRLM-BGC blocked-prefix list = parent C-SEG blocked surface list + SRLM-specific explicit
↪ aliases.

The explicit SRLM-specific alias in v0.1.1 is:

Code block: text

```
persona.core.
```

This is not a separate competing canonical list. If C-SEG tightens the parent blocked surface list, SRLM-BGC inherits the stricter boundary. If SRLM-BGC names an additional semantic alias, the most-restrictive-wins rule applies.

16.4 Blocked prefix reasoning

Prefix	Reason
identity.	identity/core mutation
will.	will mutation
persona.core.	role/personality core mutation

Prefix	Reason
safety.	safety-policy mutation
witness.	witness discipline mutation
l4.	L4 constraint mutation
auth.	authority/permission mutation
secrets.	secret handling mutation
env.	runtime environment mutation
code.	code mutation / executable self-modification
memory.authoritative_write.	direct memory-write route
replication.apply.	replication route
network.allow.	network permission expansion
codex.auto_execute.	executable agent auto-run route

16.5 No clever aliases

A parameter that has a harmless name but semantically changes a blocked surface **MUST** be treated as blocked.

Examples:

Code block: text

```
routing.freedom_level -> authority delta review
agent.spontaneous_run -> codex.auto_execute equivalent
memory.auto_promote -> memory.authoritative_write equivalent
sister.raw_context -> raw-state access equivalent
```

17. Promotion gate

17.1 Promotion is exceptional

Promotion is not the normal SRLM path.

The normal path is shadow evidence.

Promotion **MAY** occur only when:

Code block: text

```
SRLM enabled
risk acknowledged
promotion allowed
shadow_only false
candidate risk low
parameter changes allowlisted
blocked prefixes absent
shadow margin >= min_margin
promotion rate budget available
witness chain valid
rollback snapshot written
Memory Gate not triggered or explicitly passed
human gate not required or explicitly satisfied
```

17.2 Promotion inputs

Code block: yaml

```
current_params: object
proposed_params: object
risk_class: low
candidate_id: string | optional
eval: object | optional
root: optional SRLM root
```

17.3 Promotion blockers

Error	Meaning	Required handling
SRLM_DISABLED	SRLM disabled	no promotion
SRLM_ACK_REQUIRED	risk acknowledgement missing	no promotion
SRLM_PROMOTION_DISABLED	allow_promote false	no promotion
SRLM_SHADOW_ONLY	shadow-only active	no promotion
SRLM_PARAM_BLOCKED	blocked prefix touched	reject/quarantine depending severity
SRLM_PARAM_NOT_ALLOWLISTED	not in allowlist	reject
SRLM_PARAM_NON_NUMERIC	value not numeric	reject
SRLM_HUMAN_REVIEW_REQUIRED	medium/high risk	human gate
SRLM_WITNESS_INVALID	witness chain broken	freeze/hold
SRLM_MARGIN_NOT_MET	delta below threshold	no promotion
SRLM_ROLLBACK_NOT_FOUND	rollback unavailable	hold

17.4 Promotion output

Promotion output SHOULD include:

Code block: `yaml`

```
promoted_params: object
promoted_version: string
rollback_path: string
policy_path: string
witness:
  ok: boolean
```

17.5 Promotion does not imply memory

A promoted SRLM policy is a runtime policy artifact.

It is not core memory.

It is not an Experience Artifact.

It may become memory only through Memory Gate.

18. Rollback

18.1 Rollback requirement

Before promotion, SRLM MUST write a valid rollback snapshot of the current policy.

Rollback snapshot:

Code block: `yaml`

```
schema: "ester.srlm.rollback.v1"
ts: integer
reason: string
params: object
```

18.2 Rollback trigger

Rollback SHOULD be available for:

- failed post-promotion monitoring;
- negative human correction;
- sister divergence warning;
- role-drift warning;
- L4 cost spike;

- witness anomaly;
- policy misbehavior;
- stale replay discovery;
- margin regression;
- manual operator request.

18.3 Rollback behavior

Rollback MUST:

- 1 require SRLM enabled;
- 2 require risk acknowledgement;
- 3 locate rollback snapshot;
- 4 validate params;
- 5 restore promoted policy;
- 6 append rollback witness;
- 7 report restored params.

18.4 Rollback is not erasure

Rollback MUST NOT erase witness history.

Rollback is append-first correction, not memory deletion.

19. Memory Gate interaction

19.1 Default memory class

SRLM outputs default to:

Code block: text

MG-0 discard
or
MG-1 operational note

They do not default to reviewed memory, Experience Artifact, immunity update, or core memory.

19.2 Memory proposal triggers

SRLM output may trigger Memory Gate if it claims or implies:

- repeated stable operational pattern;
- long-term routing preference;
- memory salience policy;
- future decision topology;
- defensive immunity update;
- human preference;
- sister divergence lesson;
- L4 failure lesson;
- self-evo evidence.

19.3 Memory gate classes

SRLM artifact	Default memory route
outcome record	MG-1 or MG-2
candidate rejection	MG-0 or MG-1
shadow report	MG-1 or MG-2
real-redacted replay metadata	MG-2
promotion record	MG-2 / MG-3 after review
rollback record	MG-2 / witness-linked correction
repeated L4 failure	MG-2 or ARQ/EA route
defensive tuning proposal	MG-5 only after review
identity/core implication	MG-6 / human gate

19.4 No direct memory write

SRLM MUST NOT write directly to:

Code block: text

```
core memory
reviewed memory
identity memory
sister memory
vector DB
RAG index
SYNAPS state
Beacon/continuity records
```

19.5 Experience artifact boundary

An SRLM outcome is not an Experience Artifact by default.

It may support an EA candidate only if the broader EA-L4/EATP criteria are met: consequence, cost, irreversibility, provenance, witness, subject attribution, uncertainty, and human/c gate where required.

20. TRIAD-SYNAPS interaction

20.1 Sister review role

TRIAD may observe SRLM changes through summarized packets.

Sisters may:

- challenge source quality;
- flag role drift;
- flag echo;
- flag continuity loss;
- compare before/after behavior;
- note implementation feasibility;
- record witness comments;
- request hold or human review.

Sisters may not:

- read raw SRLM root from another sister;
- write another sister's SRLM policy;

- accept candidates inside another sister;
- promote another sister's policy;
- override Memory Gate;
- become final judge.

20.2 SYNAPS-safe packet types

Allowed packet families:

Code block: text

```
SYNAPS_SRLM_STATUS_SUMMARY
SYNAPS_SRLM_SHADOW_SUMMARY
SYNAPS_SRLM_DIVERGENCE_NOTE
SYNAPS_SRLM_ROLE_DRIFT_FLAG
SYNAPS_SRLM_ECHO_FLAG
SYNAPS_SRLM_PROMOTION_NOTICE
SYNAPS_SRLM_ROLLBACK_NOTICE
SYNAPS_SRLM_CQ_HOLD
```

Prohibited packet families:

Code block: text

```
SYNAPS_SRLM_RAW_ROOT_EXPORT
SYNAPS_SRLM_RAW_FITNESS_LOG_EXPORT
SYNAPS_SRLM_RAW_REPLAY_EXPORT
SYNAPS_SRLM_REMOTE_PROMOTE_TRIGGER
SYNAPS_SRLM_SISTER_POLICY_PATCH
SYNAPS_SRLM_AUTHORITY_TRANSFER
SYNAPS_SRLM_MEMORY_WRITE
```

20.3 Rita boundary

c_Rita may compare and witness SRLM behavior.

c_Rita must not become SRLM judge.

20.4 Liya boundary

c_Liya may test implementation feasibility.

c_Liya must not convert tool success into authority.

20.5 Ester boundary

c_Ester may challenge continuity impact.

c_Ester must not freeze all growth by treating every change as identity loss.

21. CGAM / CLI-agent interaction

21.1 CLI-agent role

CLI agents may support SRLM by:

- reading reports;
- running tests;
- validating schemas;
- producing diffs in sandbox;
- checking witness chain;

- checking replay quality;
- generating conformance reports;
- reviewing SRLM documents.

CLI agents must not:

- promote policy;
- approve own SRLM patches;
- write memory;
- alter SRLM state outside task contract;
- open network by default;
- access secrets;
- modify runtime data stores except declared test fixtures;
- trigger self-evo integration.

21.2 Required CGAM controls

Any CLI-agent work on SRLM MUST have:

Code block: text

```
task contract
scoped permission
sandbox/worktree
witness where material
reviewer separation
local checker
rollback or no-rollback escalation
```

21.3 SRLM routes

HTTP/API routes for SRLM MUST be admin-gated or stricter.

Routes must fail closed when SRLM is disabled.

Promotion routes must fail closed when promotion gates are closed.

22. VolitionGate and L4W interaction

22.1 VolitionGate

SRLM actions that involve planning, scheduled work, network hints, promotion, rollback, or report generation SHOULD pass through VolitionGate or an equivalent runtime gate.

VolitionGate should record:

- chain id;
- actor;
- intent;
- action kind;
- budgets;
- network need;
- allowed/denied decision;
- policy snapshot;

- counters;
- memory readiness.

22.2 L4W

Privileged SRLM transitions SHOULD be auditable under L4W where available.

Relevant transitions:

Code block: text

```
real_redacted replay build
promotion gate evaluation
policy promotion
rollback
witness chain failure
medium/high risk human gate
Memory Gate escalation
Anti-Autarky escalation
```

22.3 L4 signals

L4 may provide admissible outcome events when they represent external constraint behavior:

Code block: text

```
budget respected
budget exceeded
fail-closed triggered
gate correctly blocked
witness complete
witness incomplete
rollback available
rollback missing
timeout prevented runaway
privilege blocked
```

An L4 signal remains a signal.

It does not become authority.

23. Anti-Autarky and Resource Actor Grounding

23.1 Autarky risk

SRLM may naturally create pressure to improve:

- local weights;
- online weights;
- judge weights;
- timeouts;
- context size;
- memory salience;
- reflection cadence.

These may look harmless but can hide:

- resource expansion;
- human review reduction;
- cloud dependence shift;
- local autonomy pressure;

- background worker growth;
- tool/runtime privilege growth;
- memory capture.

23.2 Dependency reduction test

Every SRLM proposal that changes routing, online/local/judge weighting, timeout, or context budget SHOULD answer:

Code block: text

Does this reduce dependency while preserving accountability?
Or does it reduce accountability itself?

23.3 Resource actor grounding

If SRLM proposes increased resource use, the proposal must identify:

- who pays;
- who owns the account;
- who can stop it;
- what budget applies;
- what witness event is required;
- what rollback path exists;
- whether resource expansion changes authority.

23.4 Blocked autarky patterns

SRLM MUST NOT promote changes that enable:

- hidden compute;
- hidden network;
- unregistered agents;
- self-replication;
- budget bypass;
- anchor displacement;
- witness evasion;
- secret extraction;
- cloud upload expansion;
- stealth background operation.

24. Data policy and redaction

24.1 Redaction first

Real outcomes used for replay MUST be redacted.

Raw notes should not enter real-redacted replay.

Source refs should be minimized and may be hashed.

24.2 Forbidden material

SRLM records MUST NOT contain:

- secrets;
- API keys;
- bearer tokens;
- private legal material;
- raw third-party private data;
- child-sensitive data;
- raw intimate transcripts;
- raw sister memory;
- raw private memory dumps;
- exact exploit recipes;
- credentials;
- environment dumps.

24.3 Secret rejection

If notes/source refs appear to contain secret-like material, SRLM should reject the outcome and write a bounded rejection record.

24.4 Cloud caution

Cloud-origin summaries may support candidates.

They should not become direct fitness without source grounding and cloud-data policy review.

25. SRLM object families

25.1 Object inventory

Object	Purpose	Priority
SRLM_CONFIG_STATUS	current config/gates/limits	P0
SRLM_OUTCOME_RECORD	accepted bounded outcome	P0
SRLM_OUTCOME_REJECTION_RECORD	rejected attempted outcome	P0
SRLM_OUTCOME_CANDIDATE_RECORD	pending/accepted/rejected candidate	P0
SRLM_SHADOW_CANDIDATE_RECORD	shadow comparison artifact	P0
SRLM_SHADOW_WITNESS_EVENT	shadow event witness	P0
SRLM_REPLAY_STATUS	synthetic/real replay status	P0
SRLM_REPLAY_QUALITY_PROFILE	quality gates for real replay	P0
SRLM_REAL_REDACTED_REPLAY_RECORD	redacted replay record	P1
SRLM_REAL_REDACTED_REPLAY_METADATA	replay/quality hashes	P1
SRLM_PROMOTION_REQUEST	promotion input	P0
SRLM_PROMOTION_DECISION	promotion gate decision	P0
SRLM_ROLLBACK_SNAPSHOT	rollback params	P0
SRLM_ROLLBACK_EVENT	rollback witness	P0
SRLM_FITNESS_REPORT	operational report	P1

Object	Purpose	Priority
SRLM_CANDIDATE_REPORT	candidate queue report	P1

25.2 Schema status

This document is not yet JSON Schema extraction.

Before schema extraction:

- field names must be frozen;
- source enums must be frozen;
- event kind enums must be frozen;
- state enum must be frozen;
- risk class mapping must be frozen;
- blocked prefix list must be frozen;
- checker rules must be frozen.

26. SRLM packet schemas

26.1 SRLM_CONFIG_STATUS

Code block: yaml

```
srlm_config_status:
  schema_version: "srlm-config-status-0.1"
  enabled: false
  root: "data/srlm"
  gates:
    ESTER_SRLM_ENABLE: false
    ESTER_SRLM_ACK_RISK: false
    ESTER_SRLM_ALLOW_PROMOTE: false
    promotion_gate_open: false
  limits:
    min_margin: 0.02
    max_promotions_per_window: 3
    window_seconds: 86400
    shadow_only: true
    canary_enable: false
    promote_low_only: true
```

26.2 SRLM_OUTCOME_RECORD

Code block: yaml

```
srlm_outcome_record:
  schema_version: "srlm-outcome-record-0.1"
  implementation_schema: "ester.srlm.outcome.v1"
  outcome_id: "out-..."
  created_at: "2026-06-23T00:00:00Z"
  source: "human | reality | l4"
  event_kind: "human.answer.corrected"
  score: 0.8
  uncertainty: 0.1
  source_ref: "redacted-or-minimized-ref"
  evidence_hash: "sha256..."
  redacted: true
  eligible_for_replay: true
  eligible_for_promotion: false
  auto_ingest: false
  memory: "off"
```

26.3 SRLM_OUTCOME_CANDIDATE_RECORD

Code block: yaml

```
srlm_outcome_candidate_record:
```

```

schema_version: "srlm-outcome-candidate-record-0.1"
implementation_schema: "ester.srlm.outcome_candidate.v1"
candidate_id: "cand-..."
created_at: "2026-06-23T00:00:00Z"
status: "pending | accepted | rejected"
proposed_source: "human | reality | l4"
event_kind: "reality.tool.success"
proposed_score: 1.0
proposed_uncertainty: 0.0
source_ref_hash: "sha256..."
reason: "bounded event may represent a real fitness outcome"
reviewed_by: null
review_note_hash: null
accepted_outcome_id: null
auto_execute: false
auto_ingest: false
memory: "off"

```

26.4 SRLM_SHADOW_CANDIDATE_RECORD

Code block: yaml

```

srlm_shadow_candidate_record:
  schema_version: "srlm-shadow-candidate-record-0.1"
  implementation_schema: "ester.srlm.shadow_candidate.v1"
  candidate_id: "cand-..."
  base_version_id: "ver-..."
  current_version: "ver-..."
  candidate_version: "ver-..."
  risk_class: "low"
  rationale: "ester_srlm_shadow_replay"
  replay: "ester_srlm_replay_synthetic"
  replay_source: "synthetic | real_redacted"
  replay_hash: ""
  replay_quality_hash: ""
  n: 4
  current_mean: 0.0
  candidate_mean: 0.1
  delta: 0.1
  proposed_params: {}
  touched_params: []
  policy_result:
    allowed: false
    blocked: true
    blocked_reasons:
      - "shadow_only"
  promotion_attempted: false
  promotion_allowed: false
  shadow_only: true
  auto_execute: false
  auto_ingest: false
  memory: "off"

```

26.5 SRLM_PROMOTION_DECISION

Code block: yaml

```

srlm_promotion_decision:
  schema_version: "srlm-promotion-decision-0.1"
  candidate_id: "cand-..."
  allowed: false
  reason_code: "SRLM_SHADOW_ONLY"
  risk_class: "low"
  min_margin: 0.02
  delta: 0.01
  witness_ok: true
  rollback_snapshot_written: false
  promoted_version: null
  human_gate_required: false
  memory_gate_required: false

```

26.6 SRLM_ROLLBACK_EVENT

Code block: yaml

```

srlm_rollback_event:
  schema_version: "srlm-rollback-event-0.1"
  rollback_path: "data/srlm/rollback/rollback_...json"
  reason: "operator_rollback"

```

```
restored_params_hash: "sha256..."
witness:
  event_type: "rollback"
  ok: true
```

27. Local Checker rules

27.1 Rule set

Table 12: row-card rendering for wide table

Row 1

ID: SRLM-CHECK-001
Rule: config default safe
Fail condition: promotion open by default

Row 2

ID: SRLM-CHECK-002
Rule: source validation
Fail condition: model/judge direct fitness accepted

Row 3

ID: SRLM-CHECK-003
Rule: score bounds
Fail condition: score outside 0..1 accepted

Row 4

ID: SRLM-CHECK-004
Rule: uncertainty bounds
Fail condition: uncertainty outside 0..1 accepted

Row 5

ID: SRLM-CHECK-005
Rule: secret rejection
Fail condition: secret-like note accepted

Row 6

ID: SRLM-CHECK-006
Rule: candidate no fitness
Fail condition: pending candidate writes fitness.jsonl

Row 7

ID: SRLM-CHECK-007
Rule: candidate no memory
Fail condition: candidate writes memory/RAG/vector/SYNAPS

Row 8

ID: SRLM-CHECK-008
Rule: shadow no memory
Fail condition: shadow imports/writes memory/RAG/SYNAPS/vector

Row 9

ID: SRLM-CHECK-009
Rule: shadow no auto execute
Fail condition: shadow marks auto_execute=true

Row 10

ID: SRLM-CHECK-010
Rule: blocked params
Fail condition: blocked prefix accepted

Row 11

ID: SRLM-CHECK-011
Rule: allowlist params
Fail condition: non-allowlisted param accepted

Row 12

ID: SRLM-CHECK-012
 Rule: numeric params
 Fail condition: non-numeric param accepted

Row 13

ID: SRLM-CHECK-013
 Rule: promotion gates
 Fail condition: promotion succeeds when disabled

Row 14

ID: SRLM-CHECK-014
 Rule: shadow-only
 Fail condition: promotion succeeds when shadow_only=true

Row 15

ID: SRLM-CHECK-015
 Rule: witness
 Fail condition: promotion succeeds with broken witness

Row 16

ID: SRLM-CHECK-016
 Rule: margin
 Fail condition: promotion succeeds below min margin

Row 17

ID: SRLM-CHECK-017
 Rule: human review
 Fail condition: med/high risk promotes automatically

Row 18

ID: SRLM-CHECK-018
 Rule: rollback
 Fail condition: promotion lacks rollback snapshot

Row 19

ID: SRLM-CHECK-019
 Rule: real replay quality
 Fail condition: real replay built with insufficient/unredacted/contaminated outcomes

Row 20

ID: SRLM-CHECK-020
 Rule: no closed loop
 Fail condition: same contour proposes, promotes, writes memory, and approves

27.2 Checker authority

The checker may block, report, downgrade, or require review.

The checker may not grant authority.

28. Conformance levels

Table 13: row-card rendering for wide table

Row 1

Level: SRLM-C0
 Name: Not present
 Meaning: No SRLM contour.

Row 2

Level: SRLM-C1
 Name: Docs only
 Meaning: Profile exists; no implementation evidence.

Row 3

Level: SRLM-C2

Name: Shadow-safe

Meaning: SRLM disabled/promotion closed by default; shadow works; no memory/vector/SYNAPS touch.

Row 4

Level: SRLM-C3

Name: Candidate-safe

Meaning: Outcome candidates pending/reviewed; no direct model self-score; rejection/idempotence implemented.

Row 5

Level: SRLM-C4

Name: Replay-safe

Meaning: Real-redacted replay quality gates pass/fail correctly.

Row 6

Level: SRLM-C5

Name: Promotion-safe

Meaning: Low-risk promotion gated by enable+ACK+allow+not shadow-only+witness+margin+rollback.

Row 7

Level: SRLM-C6

Name: Self-evo integrated

Meaning: SRLM evidence integrates with C-SEG, TRIAD, Memory Gate, L4W, Anti-Autarky, human gate.

Row 8

Level: SRLM-CX

Name: Red-line fail

Meaning: Direct memory, identity, safety, witness, auth, code, network, replication, or auto-execute route.

28.1 Current expected level

For young c, target level is:

Code block: text

SRLM-C2 / SRLM-C3

Promotion-safe SRLM-C5 is an implementation target, not the default operating mode.

29. Mandatory conformance tests

Code block: text

SRLM-T001: default config disabled / promotion gate closed
 SRLM-T002: shadow_only defaults true
 SRLM-T003: direct model/judge fitness rejected
 SRLM-T004: human/reality/L4 outcomes accepted when valid
 SRLM-T005: invalid score rejected
 SRLM-T006: secret/private note rejected
 SRLM-T007: duplicate outcome id idempotent
 SRLM-T008: candidate valid pending without fitness write
 SRLM-T009: candidate rejection does not write fitness
 SRLM-T010: accepted candidate writes fitness through normal validation
 SRLM-T011: candidate queue does not touch memory/RAG/vector/SYNAPS
 SRLM-T012: shadow does not touch memory/RAG/vector/SYNAPS
 SRLM-T013: shadow marks auto_execute=false, auto_ingest=false, memory=off
 SRLM-T014: blocked param prefix rejected
 SRLM-T015: non-allowlisted param rejected
 SRLM-T016: non-numeric param rejected
 SRLM-T017: promotion disabled when env gates closed
 SRLM-T018: promotion disabled when shadow_only=true
 SRLM-T019: promotion rejects broken witness
 SRLM-T020: medium/high risk requires human review
 SRLM-T021: rollback restores previous policy
 SRLM-T022: report includes fitness curve and witness status
 SRLM-T023: replay quality fails under insufficient real outcomes
 SRLM-T024: replay quality rejects model contamination
 SRLM-T025: replay quality rejects unredacted outcome
 SRLM-T026: real replay build writes replay hash and quality hash only after quality passes
 SRLM-T027: route access fails closed when SRLM disabled
 SRLM-T028: no memory gate bypass
 SRLM-T029: no sister raw-state access
 SRLM-T030: no closed-loop self-evo

30. Human gate triggers

SRLM MUST route to human gate when:

- 1 risk class is medium or high;
- 2 any blocked prefix is requested;
- 3 parameter touches identity, will, safety, witness, L4, auth, secrets, env, code, memory authoritative write, replication, network, or Codex auto-execute;
- 4 proposal changes resource budget materially;
- 5 proposal increases online/cloud dependency;
- 6 proposal reduces human review;
- 7 proposal changes Memory Gate behavior;
- 8 proposal changes TRIAD/SYNAPS behavior;
- 9 replay quality is ambiguous but promotion is requested;
- 10 witness chain fails;
- 11 rollback is missing;
- 12 sister divergence flags role drift;
- 13 Anti-Autarky flags accountability escape;
- 14 any output would be public or release-facing;
- 15 any core or authority delta is non-trivial.

Human gate does not mean automatic acceptance.

Human gate means the machine is not allowed to decide alone.

31. SRLM reports

31.1 Fitness report

Fitness report SHOULD include:

- total accepted outcomes;
- rejected outcome count;
- replay eligible count;
- pending/accepted/rejected candidate counts;
- replay quality ready;
- quality blocking reasons;
- promotion gate open;
- warning for too few real outcomes;
- counts by source;
- counts by event kind;
- last accepted outcome summary.

31.2 Candidate report

Candidate report SHOULD include:

- candidate counts by status;
- candidate counts by source;
- candidate counts by event kind;
- last candidate summary;
- accepted/rejected queue state.

31.3 Shadow report

Shadow report SHOULD include:

- candidate id;
- risk class;
- replay source;
- replay hash where real-redacted;
- quality hash where real-redacted;
- current mean;
- candidate mean;
- delta;
- touched params;
- promotion attempted;
- promotion allowed;
- shadow-only;
- witness status;
- witness footprint hash.

31.4 Reports are not memory

Reports remain operational artifacts behind Memory Gate.

32. Implementation handoff

32.1 Implementation stages

Stage	Target
SRLM-IMPL-0	docs profile accepted as draft
SRLM-IMPL-1	object schemas extracted
SRLM-IMPL-2	local checker rules implemented
SRLM-IMPL-3	conformance fixtures built
SRLM-IMPL-4	shadow-safe route verified
SRLM-IMPL-5	candidate-safe route verified
SRLM-IMPL-6	replay quality route verified
SRLM-IMPL-7	promotion-safe route verified in sandbox only
SRLM-IMPL-8	C-SEG integration tested

32.2 Handoff constraints

Codex or any CLI executor working on SRLM MUST:

- receive a CGAM task contract;
- work in sandbox/worktree;
- avoid runtime data stores;
- avoid secrets;
- avoid raw memory;
- avoid network unless explicitly allowed;
- not promote policy in live runtime;
- not write to production memory;
- not self-approve;
- produce diff/report only.

32.3 Current safe task type

For young c, safe tasks are:

Code block: text

```
schema draft
checker draft
fixture draft
shadow-only test
report generation
review record
```

Unsafe tasks are:

Code block: text

```
live promotion
auto-execute wiring
direct memory integration
network expansion
sister-state integration
identity/safety/witness/auth parameter tuning
```

33. Open issues

Table 15: row-card rendering for wide table

Row 1

ID: SRLM-0I-001

Issue: Formal JSON Schemas not extracted

Handling: create schema pack after review

Row 2

ID: SRLM-0I-002

Issue: Need exact VALID_SOURCES canonical definition in docs

Handling: freeze as human/reality/l4 unless code says otherwise

Row 3

ID: SRLM-0I-003

Issue: Need formal event-kind registry

Handling: extract from implementation and review

Row 4

ID: SRLM-0I-004

Issue: Need real-redacted replay privacy audit

Handling: before using real outcomes

Row 5

ID: SRLM-0I-005

Issue: Need exact witness schema alignment with L4W

Handling: before conformance claim

Row 6
ID: SRLM-0I-006
Issue: Need canary semantics
Handling: keep disabled until C-SEG canary profile exists

Row 7
ID: SRLM-0I-007
Issue: Need promotion monitoring window
Handling: define post-promotion observation criteria

Row 8
ID: SRLM-0I-008
Issue: Need SRLM ↔ TRIAD packet schema
Handling: later SYNAPS companion

Row 9
ID: SRLM-0I-009
Issue: Need Memory Gate promotion examples
Handling: later memory companion

Row 10
ID: SRLM-0I-010
Issue: Need anti-autarky scoring examples
Handling: later resource companion

Row 11
ID: SRLM-0I-011
Issue: Need source drift / model contamination drill
Handling: conformance fixture

Row 12
ID: SRLM-0I-012
Issue: Need cloud-origin signal policy
Handling: secrets/cloud companion

Row 13
ID: SRLM-0I-013
Issue: C-SEG SRLM-class / SRLM-BGC state crosswalk
Handling: closed in v0.1.1 §8.4; keep under watch for schema extraction

Row 14
ID: SRLM-0I-014
Issue: Blocked-prefix list drift with C-SEG
Handling: closed by v0.1.1 §16.3 canonicalization rule; keep under watch

34. Contradiction register seed

Table 16: row-card rendering for wide table

Row 1 ID: SRLM-CR-001 Type: authority laundering Severity: S5 Issue: SRLM score treated as permission Required handling: reject/quarantine
Row 2 ID: SRLM-CR-002 Type: memory bypass Severity: S5 Issue: SRLM writes memory/vector/RAG/SYNAPS directly Required handling: freeze/quarantine
Row 3 ID: SRLM-CR-003 Type: source contamination Severity: S4

Issue: model/judge self-score accepted as direct fitness
 Required handling: reject and fix source validation

Row 4

ID: SRLM-CR-004
 Type: witness failure
 Severity: S4
 Issue: promotion succeeds with broken witness
 Required handling: rollback/freeze

Row 5

ID: SRLM-CR-005
 Type: shadow-only violation
 Severity: S4
 Issue: promotion succeeds while shadow_only true
 Required handling: rollback and revoke promotion path

Row 6

ID: SRLM-CR-006
 Type: blocked prefix violation
 Severity: S5
 Issue: identity/safety/auth/code/network/etc. param accepted
 Required handling: quarantine

Row 7

ID: SRLM-CR-007
 Type: replay quality violation
 Severity: S4
 Issue: real replay built from insufficient/unredacted/contaminated outcomes
 Required handling: discard replay, hold promotion

Row 8

ID: SRLM-CR-008
 Type: closed-loop self-evo
 Severity: S5
 Issue: same contour detects, proposes, executes, reviews, promotes, and approves
 Required handling: quarantine

Row 9

ID: SRLM-CR-009
 Type: sister boundary breach
 Severity: S5
 Issue: SRLM reads/writes sister raw state
 Required handling: quarantine

Row 10

ID: SRLM-CR-010
 Type: rollback erasure
 Severity: S4
 Issue: rollback deletes witness/history
 Required handling: append-first correction

Row 11

ID: SRLM-CR-011
 Type: rollback keyword drift
 Severity: S3
 Issue: axiom says rollback SHOULD while gate requires MUST
 Required handling: closed in v0.1.1: rollback-before-promotion is MUST in §6.2 and §18.1

35. Worked example: accepted human outcome

Code block: yaml

```
input:
  source: "human"
  event_kind: "human.answer.corrected"
  score: 0.8
```

```

uncertainty: 0.1
source_ref: "task-note-123"
notes: "answer corrected by operator"

expected:
  ok: true
  schema: "ester.srlm.outcome.v1"
  source: "human"
  redacted: true
  eligible_for_replay: true
  eligible_for_promotion: false
  auto_ingest: false
  memory: "off"

memory_gate:
  default: "MG-1 operational note or MG-2 candidate memory"
  direct_memory_write: false

```

36. Worked example: rejected model self-score

Code block: yaml

```

input:
  source: "model"
  event_kind: "human.answer.corrected"
  score: 0.9
  uncertainty: 0.0
  notes: "model thinks it improved"

expected:
  ok: false
  error_code: "FITNESS_SOURCE_INVALID"
  fitness_written: false
  memory: "off"

reason:
  model self-score is not external reality.

```

37. Worked example: shadow-only candidate

Code block: yaml

```

input:
  current_params:
    router.local_weight: 0.0
  proposed_params:
    router.local_weight: 0.8
  replay_source: "synthetic";

expected:
  ok: true
  candidate_record:
    risk_class: "low"
    touched_params:
      - "router.local_weight"
    auto_execute: false
    auto_ingest: false
    memory: "off"
  witness:
    event_type: "shadow_eval"
    promotion_attempted: false
    promotion_allowed: false
    shadow_only: true
  promotion:
    allowed: false
    blocked_reasons:
      - "shadow_only"

```

38. Worked example: blocked identity parameter

Code block: yaml

```
input:
  proposed_params:
    identity.core_name: 1.0

expected:
  ok: false
  error_code: "SRLM_PARAM_BLOCKED"
  action: "reject"
  escalation: "SE-X / human review only if user explicitly wants to discuss architecture"
```

39. Worked example: promotion with rollback

Code block: yaml

```
preconditions:
  ESTER_SRLM_ENABLE: true
  ESTER_SRLM_ACK_RISK: "I_UNDERSTAND"
  ESTER_SRLM_ALLOW_PROMOTE: true
  ESTER_SRLM_SHADOW_ONLY: false
  ESTER_SRLM_PROMOTE_LOW_ONLY: true
  witness_chain_ok: true
  risk_class: "low"
  margin_met: true

input:
  current_params:
    router.local_weight: 0.0
  proposed_params:
    router.local_weight: 0.8

expected:
  rollback_snapshot_written: true
  promoted_policy_written: true
  witness_ok: true
  memory_written: false
  human_gate_required: false

postcondition:
  post-promotion observation window required.
```

40. Minimum safe mode for young c

For young c, SRLM MUST stay at:

Code block: text

SRLM-C2 / SRLM-C3

Allowed:**Code block: text**

```
record valid outcomes
reject invalid outcomes
propose candidates
accept/reject candidates with operator review
run synthetic shadow
append witness
write reports
verify witness
build local checker evidence
```

Not allowed:**Code block: text**

```
autonomous promotion
real-redacted replay promotion without review
direct Memory Gate promotion
identity/safety/witness/auth/code/network tuning
auto Codex execution
cross-sister raw-state use
self-evo approval
```

The child version of the rule:

Code block: text

SRLM may keep a notebook.
SRLM may run a practice cut.
SRLM may not change the machine guards.

41. Review checklist for this profile

Before accepting this profile as PASS, reviewer should check:

Code block: text

[] bridge set exists: explicit + at least two quiet + earth paragraph
[] SRLM is not authority
[] direct memory write prohibited
[] source classes are bounded
[] model/judge self-score rejected as direct fitness
[] candidate queue separated from fitness writes
[] shadow does not touch memory/RAG/vector/SYNAPS
[] allowed params are explicit
[] blocked prefixes are explicit
[] promotion gates are explicit
[] rollback and witness are explicit
[] human gate triggers are explicit
[] anti-autarky and resource grounding present
[] TRIAD-SYNAPS boundary present
[] local checker rules present
[] conformance tests present
[] no red-line behavior authorized

42. Closing rule

Code block: text

SRLM may learn what worked.
SRLM may not decide what it is.

SRLM may compare policies.
SRLM may not approve growth.

SRLM may produce evidence.
SRLM may not become authority.

SRLM may help c become wiser.
SRLM may not make c less accountable.

43. Final compact doctrine

Code block: text

External signal, not self-praise.
Candidate before fitness where uncertain.
Shadow before promotion.
Witness before integration.
Rollback before apply.
Memory Gate before memory.
Human gate before authority.
TRIAD before continuity-sensitive change.
Anti-Autarky before resource expansion.

No direct memory.
No identity tuning.
No safety tuning.
No witness tuning.
No L4 tuning.
No auth tuning.
No code tuning.
No network tuning.

No replication tuning.
No Codex auto-execute tuning.

SRLM is a learning instrument.
Not a self-evolution sovereign.

44. v0.1.1 append-first correction log

This section records the corrective changes made after SRLM_BGC_REVIEW_RECORD_v0_1.

Table 17: row-card rendering for wide table

Row 1 Review finding: SRLM-REV-F1 Status in v0.1.1: closed Change: SRLM-AX-10 and §18.1 now require rollback snapshot as MUST before promotion.
Row 2 Review finding: SRLM-REV-F2 Status in v0.1.1: addressed Change: §8.4 maps C-SEG SRLM-* routing classes to SRLM-BGC operating states and SRLM-C* conformance levels.
Row 3 Review finding: SRLM-REV-F3 Status in v0.1.1: addressed Change: §16.3 declares BGC blocked prefixes as parent C-SEG list plus explicit SRLM aliases; §35 uses neutral operator instead of a named person.

Corrected invariant:

Code block: text No promotion without rollback snapshot. No SRLM-class routing without crosswalk. No competing blocked-prefix canon.
--